Software Quality Assurance

Dr. Khaled Negm

Lab 7

Page Object Model (POM) & Page Factory

2 Agenda

Theory

- What is POM?
- Why POM?
- Advantages of POM

Implementation

- How to implement POM?
- Guru99 complete example

Page Factory

- What is Page Factory?
- AjaxElementLocatorFactory

Exercises + Solutions

- Ex 1 — Simple POM
- Ex 2 — Page Factory
- Ex 3 ★ Multi-Page POM
- Ex 4 ★ Ajax Factory

Theory

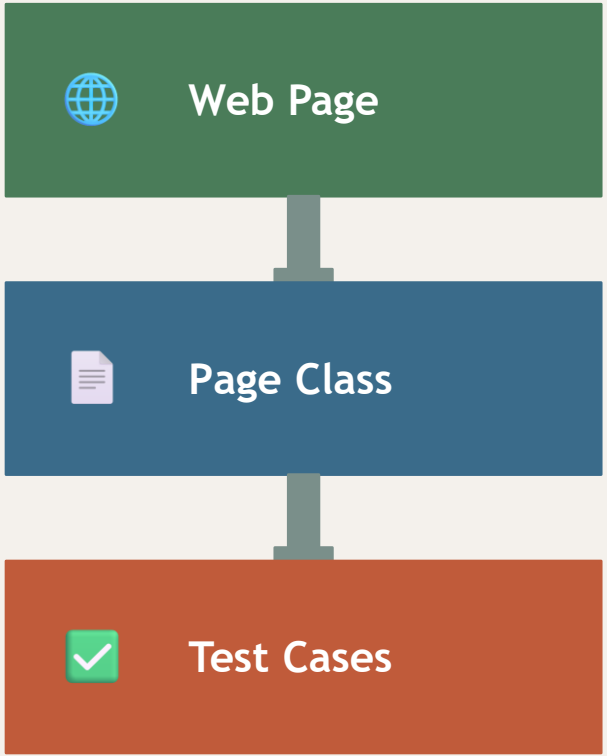
What POM is, why it matters, and its advantages

03

4 What is Page Object Model?

Design pattern for test automation

POM creates an Object Repository for web UI elements — reducing code duplication and improving test maintenance. For each web page there is a corresponding Page Class that identifies WebElements and contains methods to operate on them.



Method naming convention

Name methods after the task they perform, not how they work:

```
waitForPaymentScreenDisplay()
clickLoginButton()
enterUserCredentials(user, pwd)
verifyBankTitlePresent()
```

5 Why POM?

❌ The Problem

- Script finds elements and performs operations directly
- 10 scripts share the same page element
- Element changes require editing 10 files
- Time-consuming & error-prone maintenance
- Test suite grows, complexity explodes

✅ POM Solution

- Separate class file per page of the app
- All scripts share one page class
- Change element → update 1 class only
- Clean, readable, reusable test code
- Scales gracefully as test suite grows

6 Advantages of POM

Separation of Concerns

UI operations separated from verification — cleaner, easier-to-read code.

Tool Independence

Same repo works with TestNG/JUnit for functional testing AND JBehave/Cucumber for acceptance testing.

Less & Optimised Code

Reusable page methods in POM classes eliminate duplication across the entire suite.

Realistic Method Names

Methods like gotoHomePage() clearly map to UI operations — instant readability for the whole team.

Implementation

07

Simple POM → Guru99 complete example

8 Simple POM — Structure

One class per page; verification stays in @Test methods

LoginPage.java

```
By userName
By password
By loginButton
By bankTitle

setUserName(str)
setPassword(str)
clickLogin()
getBankTitle()
```

HomePage.java

```
By homePageUserName

getHomePageDashboardUserName()
```

TestGuru99.java

```
@Test method

new LoginPage(driver)
new HomePage(driver)

setUserName(...)
clickLogin()
Assert.assertTrue(...)
```

9 Complete Example — Guru99 Bank

01

Navigate to site

https://demo.guru99.com/v4/

02

Verify Login page

Check text "Guru99 Bank" is present

03

Login

UserID: mngr559943 · Password: AqajysY (valid 20 days, create new at demo.guru99.com)*

04

Verify Home page

Heading contains: Manger Id: mngr559943

10 LoginPage.java — Simple POM

LoginPage.java

```
public class LoginPage {
    WebDriver driver;

    // Locators
    By userName    = By.name("uid");
    By password    = By.name("password");
    By loginButton = By.name("btnLogin");
    By bankTitle   = By.className("barone");

    // Constructor
    public LoginPage(WebDriver driver) {
        this.driver = driver;
    }

    // Page Methods
    public void   setUserName(String s) { driver.findElement(userName).sendKeys(s); }
    public void   setPassword(String s) { driver.findElement(password).sendKeys(s); }
    public void   clickLogin()          { driver.findElement(loginButton).click(); }
    public String getBankTitle()        { return driver.findElement(bankTitle).getText(); }
}
```

11 HomePage.java + TestGuru99.java

HomePage.java

```
public class HomePage {
    WebDriver driver;
    By heading = By.xpath(
      "//table//tr[@class='heading3']");

    public HomePage(WebDriver driver) {
        this.driver = driver;
    }

    public String getHomePageDashboardUserName() {
        return driver.findElement(heading).getText();
    }
```

TestGuru99.java

```
public class TestGuru99 {
    WebDriver driver = new FirefoxDriver();
    LoginPage objLogin   = new LoginPage(driver);
    HomePage  objHome    = new HomePage(driver);

    @Test(priority=0)
    public void test_Home_Page() throws Exception {
        driver.get("http://demo.guru99.com/V4/");
    // ... continued below
    }
}
```

TestGuru99.java — continued

```
        // Verify bank title on Login page
        String sTitle = objLogin.getBankTitle();
        Assert.assertTrue(sTitle.toLowerCase().contains("guru99 bank"));

        // Login
        objLogin.setUserName("mngr559943");
        objLogin.setPassword("AqajysY");
        objLogin.clickLogin();

        // Verify Home page
        String sUser = objHome.getHomePageDashboardUserName();
        Assert.assertTrue(sUser.toLowerCase().contains("mngr559943"));
```

Page Factory

Optimised POM with @FindBy annotations

12

13

What is Page Factory?

Inbuilt optimised POM framework for Selenium WebDriver

Page Factory initialises Page objects without calling FindElement. It uses annotations and works alongside the separation of Page Object Repository from Test Methods.

```
@FindBy
```

Locates WebElement by: id · name · xpath · css · className · tagName · linkText · partialLinkText

```
PageFactory.initElements(driver, this)
```

Called in the constructor. Initialises ALL @FindBy elements at once — replaces individual findElement() calls.

14 LoginPageWithPageFactory.java

LoginPageWithPageFactory.java

```
public class LoginPageWithPageFactory {
    WebDriver driver;

    // @FindBy annotations replace By locators
    @FindBy(name = "uid")
    WebElement userNameElement;

    @FindBy(name = "password")
    WebElement passwordElement;

    @FindBy(name = "btnLogin")
    WebElement loginButtonElement;

    @FindBy(className = "barone")
    WebElement bankTitleElement;

    // Constructor — PageFactory initialises all elements
    public LoginPageWithPageFactory(WebDriver driver) {
        this.driver = driver;
        PageFactory.initElements(driver, this);  // ← key line
    }

    public void   setUserName(String s) { userNameElement.sendKeys(s); }
    public void   setPassword(String s) { passwordElement.sendKeys(s); }
    public void   clickLogin()          { loginButtonElement.click(); }
    public String getBankTitle()        { return bankTitleElement.getText(); }
}
```

15 HomePageWithPageFactory.java + Comparison

HomePageWithPageFactory.java

```
public class HomePageWithPageFactory {
    @FindBy(xpath =
      "//table//tr[@class='heading3']")
    WebElement homePageUserName;

    public HomePageWithPageFactory(WebDriver driver){
        PageFactory.initElements(driver, this);
    }

    public String getHomePageDashboardUserName() {
        return homePageUserName.getText();
    }
}
```

Simple POM vs Page Factory	
By locators	@FindBy annotations
findElement() each time	Direct element usage
No init needed	PageFactory.initElements()
More verbose	Cleaner, concise

TestGuru99WithPageFactory.java

```
// TestGuru99WithPageFactory.java
public class TestGuru99WithPageFactory {
    WebDriver driver = new FirefoxDriver();
    LoginPageWithPageFactory  objLogin = new LoginPageWithPageFactory(driver);
    HomePageWithPageFactory   objHome  = new HomePageWithPageFactory(driver);

    @Test(priority=0)
    public void test_Home_Page_Appear_Correct() throws Exception {
        driver.get("http://demo.guru99.com/V4/");
        Assert.assertTrue(objLogin.getBankTitle().toLowerCase().contains("guru99 bank"));
        objLogin.setUserName("mngr559943"); objLogin.setPassword("AqajysY"); objLogin.clickLogin();
        Assert.assertTrue(objHome.getHomePageDashboardUserName().toLowerCase().contains("mngr559943"));
    } }
```

16

AjaxElementLocatorFactory

Lazy loading concept in Page Factory

Lazy Loading

Elements are located only when they are used in an operation — not at page initialisation time.

Timeout Assignment

A configurable timeout is assigned. The wait starts only when the element is interacted with.

NoSuchElementException

If the element is not found within the timeout window, NoSuchElementException is thrown.

AJAX Performance

Ideal for AJAX-heavy pages. Reduces initial load overhead by deferring element location.

17 AjaxElementLocatorFactory — Code

PageWithAjaxFactory.java

```java
import org.openqa.selenium.support.pagefactory.AjaxElementLocatorFactory;

public class PageWithAjaxFactory {
    @FindBy(id = "submitButton")
    WebElement submitBtn;

    public PageWithAjaxFactory(WebDriver driver) {
        // Elements wait up to 15 s before throwing NoSuchElementException
        AjaxElementLocatorFactory factory = new AjaxElementLocatorFactory(driver, 15);
        PageFactory.initElements(factory, this);  // factory replaces driver
    }

    public void clickSubmit() { submitBtn.click(); }
}
```

Key Difference

`PageFactory.initElements(driver, this)` → eager: finds all elements at init
`PageFactory.initElements(factory, this)` → lazy: finds each element only when used

18 Summary

Page Object Model	Object Repository design pattern — one class per web page.
Maintainability	Single class change updates all tests that use that page.
Page Factory	Optimised POM using @FindBy annotations + PageFactory.initElements().
AjaxElementLocatorFactory	Lazy load — locates elements only on interaction, with a configurable timeout.

Exercises & Solutions

Instructions + complete working code for all 4 exercises

19

Exercise 1 — Simple POM

Implement POM with By locators for the Guru99 Bank application

1 **Create LoginPage class**
By locators for uid, password, btnLogin, bankTitle. Methods: setUserName(), setPassword(), clickLogin(), getBankTitle().

2 **Create HomePage class**
By locator for heading (By.xpath). Method: getHomePageDashboardUserName().

3 **Create TestGuru99 class**
@Test: instantiate both page classes, navigate to site, verify bank title, login, verify home page message.

4 **Run & Verify**
Execute with TestNG. All assertions should pass. Screenshot the green test result.

💡 Tips for Exercise 1

- **Keep all locators as class-level fields** — never inside methods
- Verification logic (Assert.*) belongs in the @Test method, not the page class
- Import TestNG: `import org.testng.Assert;`

Exercise 2 — Page Factory

Refactor Exercise 1 to use @FindBy annotations

1

Refactor LoginPage

Replace By locators with @FindBy(name="uid") etc. declared as WebElement fields.

2

Add initElements()

In LoginPage constructor call PageFactory.initElements(driver, this) to initialise all @FindBy elements.

3

Refactor HomePage

Replace By locator with @FindBy(xpath=...). Add PageFactory.initElements() in constructor.

4

Update Test & Compare

Update test class. Run & verify it passes. Count lines saved vs Exercise 1.

💡 Tips for Exercise 2

- @FindBy supports: id · name · xpath · css · className · tagName · linkText · partialLinkText
- Import:

```
import org.openqa.selenium.support.FindBy;
import org.openqa.selenium.support.PageFactory;
```

Exercise 3 ★ — POM with Multiple Pages

Extend POM to cover Login → New Customer → Confirmation

1 **Create NewCustomerPage**
After login, navigate to New Customer. Build page class with WebElements: Customer Name, Gender, DOB, Address, City, PIN, Mobile, Email, Password, Submit.

2 **Add form methods**
fillCustomerForm(name, dob, address, city, pin, mobile, email, pwd) and clickSubmit(). Name methods after their task.

3 **Create CustomerConfirmationPage**
Read success message (e.g. 'Customer Registered Successfully') from the result table.

4 **Write & run the test**
Login → navigate to New Customer → fill form → submit → assert confirmation message. Use all three POM classes.

💡 Tips for Exercise 3

- Use driver.findElement(By.linkText("New Customer")).click() to navigate between pages
- Each page class is self-contained — it only knows about its own elements
- The test class orchestrates the flow; page classes don't call each other

27 Exercise 4 ★ — Page Factory + AjaxElementLocatorFactory

Handle dynamic/AJAX-loaded elements with lazy loading

1 **Upgrade LoginPage**

Replace PageFactory.initElements(driver,this) with AjaxElementLocatorFactory(driver, 10). Verify Exercise 2 test still passes.

2 **Build FundTransferPage**

New page class with @FindBy for payer account, payee account, amount, description. Use AjaxElementLocatorFactory(driver, 15).

3 **Write transfer test**

Methods: setPayerAccount(), setPayeeAccount(), setAmount(), setDescription(), clickSubmit(). Test: login → Fund Transfer → fill → submit → assert.

4 **Valid vs Invalid account**

Run with a valid account AND an invalid one. Assert the error message for invalid. Document: hard NoSuchElementException vs timeout.

💡 Tips for Exercise 4

- AjaxElementLocatorFactory is ideal when elements load asynchronously after page load
- The timeout clock starts only when you interact with the element (lazy), not at page init (eager)
- For the invalid account test, wrap in try-catch or use Assert.assertFalse on the success message

✅ SOLUTION — Exercise 1 (Part 1 of 2)

LoginPage.java

```
public class LoginPage {
    WebDriver driver;
    By userName    = By.name("uid");
    By password    = By.name("password");
    By loginButton = By.name("btnLogin");
    By bankTitle   = By.className("barone");

    public LoginPage(WebDriver driver) {
        this.driver = driver;
    }
    public void setUserName(String s) {
        driver.findElement(userName).sendKeys(s); }
    public void setPassword(String s) {
        driver.findElement(password).sendKeys(s); }
    public void clickLogin() {
        driver.findElement(loginButton).click(); }
    public String getBankTitle() {
        return driver.findElement(bankTitle).getText(); }
}
```

HomePage.java

```
public class HomePage {
    WebDriver driver;
    By heading = By.xpath(
      "//table//tr[@class='heading3']");

    public HomePage(WebDriver driver) {
        this.driver = driver;
    }

    public String getHomePageDashboardUserName(){
        return driver.findElement(heading)
                     .getText();
    }
}
```

TestGuru99.java

```
public class TestGuru99 {
    WebDriver driver = new FirefoxDriver();
    LoginPage objLogin = new LoginPage(driver);
    HomePage  objHome  = new HomePage(driver);

    @BeforeMethod
    public void setUp() {
        driver.manage().timeouts().implicitlyWait(10, TimeUnit.SECONDS);
    }

    @Test(priority=0)
    public void test_Home_Page_Appear_Correct() throws Exception {
        driver.get("http://demo.guru99.com/V4/");
        Assert.assertTrue(objLogin.getBankTitle().toLowerCase().contains("guru99 bank"));
        objLogin.setUserName("mngr559943");
        objLogin.setPassword("AqajysY");
        objLogin.clickLogin();
        Assert.assertTrue(objHome.getHomePageDashboardUserName().toLowerCase().contains("mngr559943"));
    }
}
```

✅ SOLUTION — Exercise 2

LoginPageWithPageFactory.java

```
public class LoginPageWithPageFactory {
    WebDriver driver;

    @FindBy(name = "uid")
    WebElement userNameElement;

    @FindBy(name = "password")
    WebElement passwordElement;

    @FindBy(name = "btnLogin")
    WebElement loginButtonElement;

    @FindBy(className = "barone")
    WebElement bankTitleElement;

    public LoginPageWithPageFactory(WebDriver driver) {
        this.driver = driver;
        PageFactory.initElements(driver, this);
    }
    public void   setUserName(String s) { userNameElement.sendKeys(s); }
    public void   setPassword(String s) { passwordElement.sendKeys(s); }
    public void   clickLogin()          { loginButtonElement.click(); }
    public String getBankTitle()        { return bankTitleElement.getText(); }
}
```

HomePageWithPageFactory.java + Test

```
public class HomePageWithPageFactory {
    @FindBy(xpath =
      "//table//tr[@class='heading3']")
    WebElement homePageUserName;

    public HomePageWithPageFactory(WebDriver d) {
        PageFactory.initElements(d, this);
    }
    public String getHomePageDashboardUserName(){
        return homePageUserName.getText();
    }
}

// ── Test Class ──────────────────────────────
public class TestGuru99WithPageFactory {
    WebDriver driver = new FirefoxDriver();
    LoginPageWithPageFactory  objLogin =
        new LoginPageWithPageFactory(driver);
    HomePageWithPageFactory   objHome  =
        new HomePageWithPageFactory(driver);

    @Test(priority=0)
    public void test_Home_Page() throws Exception {
        driver.get("http://demo.guru99.com/V4/");
        Assert.assertTrue(objLogin.getBankTitle()
          .toLowerCase().contains("guru99 bank"));
```

TestGuru99WithPageFactory.java — continued

```
        // continued from TestGuru99WithPageFactory
        objLogin.setUserName("mngr559943");
        objLogin.setPassword("AqajysY");
        objLogin.clickLogin();
        Assert.assertTrue(
            objHome.getHomePageDashboardUserName()
                   .toLowerCase().contains("mngr559943"));
    }
}
```

✅ SOLUTION — Exercise 3 (Part 1 of 2 — Page Classes)

NewCustomerPage.java

```
public class NewCustomerPage {
    WebDriver driver;
    By custName  = By.name("name");
    By gender    = By.xpath("//input[@value='m']");
    By dob       = By.name("dob");
    By address   = By.name("addr");
    By city      = By.name("city");
    By pin       = By.name("pinno");
    By mobile    = By.name("telephoneno");
    By email     = By.name("emailid");
    By custPwd   = By.name("password");
    By submitBtn = By.name("sub");

    public NewCustomerPage(WebDriver driver) {
        this.driver = driver;
    }

    public void fillCustomerForm(
            String name, String dob, String addr,
            String city, String pin, String mobile,
            String email, String pwd) {
        driver.findElement(this.custName).sendKeys(name);
        driver.findElement(this.gender).click();
        driver.findElement(this.dob).sendKeys(dob);
        driver.findElement(this.address).sendKeys(addr);
        driver.findElement(this.city).sendKeys(city);
        driver.findElement(this.pin).sendKeys(pin);
        driver.findElement(this.mobile).sendKeys(mobile);
        driver.findElement(this.email).sendKeys(email);
        driver.findElement(this.custPwd).sendKeys(pwd);
    }
    public void clickSubmit() {
        driver.findElement(submitBtn).click();
    }
}
```

CustomerConfirmationPage.java

```
public class CustomerConfirmationPage {
    WebDriver driver;
    By confirmMsg = By.xpath(
      "//td[contains(text(),
      'Customer Registered Successfully')]");

    public CustomerConfirmationPage(
            WebDriver driver) {
        this.driver = driver;
    }

    public String getConfirmationMessage() {
        return driver.findElement(confirmMsg)
                     .getText();
    }

    public boolean isRegistrationSuccessful(){
        return getConfirmationMessage()
            .contains(
            "Customer Registered Successfully");
    }
}
```

✅ SOLUTION — Exercise 3 (Part 2 of 2 — Test Class)

TestNewCustomer.java

```
public class TestNewCustomer {
    WebDriver driver = new FirefoxDriver();
    LoginPage              objLogin    = new LoginPage(driver);
    HomePage               objHome     = new HomePage(driver);
    NewCustomerPage        objNewCust  = new NewCustomerPage(driver);
    CustomerConfirmationPage objConfirm = new CustomerConfirmationPage(driver);

    @BeforeMethod
    public void setUp() {
        driver.manage().timeouts().implicitlyWait(10, TimeUnit.SECONDS);
    }

    @Test
    public void test_Register_New_Customer() throws Exception {
        // Step 1: Login
        driver.get("http://demo.guru99.com/V4/");
        objLogin.setUserName("mngr559943");
        objLogin.setPassword("AqajysY");
        objLogin.clickLogin();

        // Step 2: Navigate to New Customer page
        driver.findElement(By.linkText("New Customer")).click();

        // Step 3: Fill and submit the form
        objNewCust.fillCustomerForm(
            "Alice Test", "01/01/1990",
            "123 Main St", "Cairo", "123456",
            "01012345678", "alice@test.com", "Test@1234");
        objNewCust.clickSubmit();

        // Step 4: Verify confirmation
        Assert.assertTrue(objConfirm.isRegistrationSuccessful(),
            "Customer registration failed!");
    }
```

✅ SOLUTION — Exercise 4 (Part 1 of 2 — Page Classes)

LoginPageWithAjax.java

```java
// Upgraded LoginPage — using AjaxElementLocatorFactory
public class LoginPageWithAjax {
    @FindBy(name = "uid")
    WebElement userNameElement;
    @FindBy(name = "password")
    WebElement passwordElement;
    @FindBy(name = "btnLogin")
    WebElement loginButtonElement;
    @FindBy(className = "barone")
    WebElement bankTitleElement;

    public LoginPageWithAjax(WebDriver driver) {
        // Lazy load with 10-second timeout
        AjaxElementLocatorFactory factory =
            new AjaxElementLocatorFactory(driver, 10);
        PageFactory.initElements(factory, this);
    }
    public void setUserName(String s) {
        userNameElement.sendKeys(s); }
    public void setPassword(String s) {
        passwordElement.sendKeys(s); }
    public void clickLogin() {
        loginButtonElement.click(); }
    public String getBankTitle() {
        return bankTitleElement.getText(); }
}
```

FundTransferPage.java

```java
public class FundTransferPage {
    @FindBy(name = "payerAcc")
    WebElement payerAccount;
    @FindBy(name = "payeeAcc")
    WebElement payeeAccount;
    @FindBy(name = "ammount")
    WebElement amount;
    @FindBy(name = "desc")
    WebElement description;
    @FindBy(name = "AccSubmit")
    WebElement submitBtn;
    @FindBy(className = "heading3")
    WebElement resultMessage;

    public FundTransferPage(WebDriver driver) {
        // Lazy load with 15-second timeout
        AjaxElementLocatorFactory factory =
            new AjaxElementLocatorFactory(driver, 15);
        PageFactory.initElements(factory, this);
    }
    public void setPayerAccount(String s) {
payerAccount.sendKeys(s); }
    public void setPayeeAccount(String s) {
payeeAccount.sendKeys(s); }
    public void setAmount(String s)       { amount.sendKeys(s); }
    public void setDescription(String s)  {
description.sendKeys(s); }
    public void clickSubmit()             { submitBtn.click(); }
    public String getResultMessage()      { return
resultMessage.getText(); }
}
```

✅ SOLUTION — Exercise 4 (Part 2 of 2 — Test Class)

TestFundTransfer.java

```
public class TestFundTransfer {
    WebDriver driver = new FirefoxDriver();
    LoginPageWithAjax objLogin      = new LoginPageWithAjax(driver);
    FundTransferPage  objTransfer   = new FundTransferPage(driver);

    @BeforeMethod
    public void setUp() {
        driver.manage().timeouts().implicitlyWait(10, TimeUnit.SECONDS);
        driver.get("http://demo.guru99.com/V4/");
        objLogin.setUserName("mngr559943");
        objLogin.setPassword("AqajysY");
        objLogin.clickLogin();
        driver.findElement(By.linkText("Fund Transfer")).click();
    }

    @Test(priority=1, description="Valid transfer")
    public void test_Valid_Fund_Transfer() {
        objTransfer.setPayerAccount("153279");
        objTransfer.setPayeeAccount("153280");
        objTransfer.setAmount("100");
        objTransfer.setDescription("Test transfer");
        objTransfer.clickSubmit();
        Assert.assertTrue(
            objTransfer.getResultMessage().contains("Fund Transfer Successfully"),
            "Transfer should have succeeded");
    }

    @Test(priority=2, description="Invalid account — expect error")
    public void test_Invalid_Fund_Transfer() {
        objTransfer.setPayerAccount("000000");   // invalid account
        objTransfer.setPayeeAccount("000001");
        objTransfer.setAmount("100");
        objTransfer.setDescription("Invalid test");
        objTransfer.clickSubmit();
        // AjaxElementLocatorFactory: waits 15 s, then NoSuchElementException
        // if success element never appears — we assert the error message instead
        Assert.assertFalse(
            objTransfer.getResultMessage().contains("Fund Transfer Successfully"),
            "Transfer should have failed for invalid account");
    }

    @AfterMethod
    public void tearDown() { driver.quit(); }
}
```

✅ SOLUTION — Exercise 4 (Part 1 of 2 — Page Classes)

LoginPageWithAjax.java

```java
// Upgraded LoginPage — using AjaxElementLocatorFactory
public class LoginPageWithAjax {
    @FindBy(name = "uid")
    WebElement userNameElement;
    @FindBy(name = "password")
    WebElement passwordElement;
    @FindBy(name = "btnLogin")
    WebElement loginButtonElement;
    @FindBy(className = "barone")
    WebElement bankTitleElement;

    public LoginPageWithAjax(WebDriver driver) {
        // Lazy load with 10-second timeout
        AjaxElementLocatorFactory factory =
            new AjaxElementLocatorFactory(driver, 10);
        PageFactory.initElements(factory, this);
    }
    public void setUserName(String s) {
        userNameElement.sendKeys(s); }
    public void setPassword(String s) {
        passwordElement.sendKeys(s); }
    public void clickLogin() {
        loginButtonElement.click(); }
    public String getBankTitle() {
        return bankTitleElement.getText(); }
}
```

FundTransferPage.java

```java
public class FundTransferPage {
    @FindBy(name = "payerAcc")
    WebElement payerAccount;
    @FindBy(name = "payeeAcc")
    WebElement payeeAccount;
    @FindBy(name = "ammount")
    WebElement amount;
    @FindBy(name = "desc")
    WebElement description;
    @FindBy(name = "AccSubmit")
    WebElement submitBtn;
    @FindBy(className = "heading3")
    WebElement resultMessage;

    public FundTransferPage(WebDriver driver) {
        // Lazy load with 15-second timeout
        AjaxElementLocatorFactory factory =
            new AjaxElementLocatorFactory(driver, 15);
        PageFactory.initElements(factory, this);
    }
    public void setPayerAccount(String s) {
payerAccount.sendKeys(s); }
    public void setPayeeAccount(String s) {
payeeAccount.sendKeys(s); }
    public void setAmount(String s)       { amount.sendKeys(s); }
    public void setDescription(String s)  {
description.sendKeys(s); }
    public void clickSubmit()             { submitBtn.click(); }
    public String getResultMessage()      { return
resultMessage.getText(); }
}
```

✅ SOLUTION — Exercise 4 (Part 2 of 2 — Test Class)

TestFundTransfer.java

```
public class TestFundTransfer {
    WebDriver driver = new FirefoxDriver();
    LoginPageWithAjax objLogin      = new LoginPageWithAjax(driver);
    FundTransferPage  objTransfer   = new FundTransferPage(driver);

    @BeforeMethod
    public void setUp() {
        driver.manage().timeouts().implicitlyWait(10, TimeUnit.SECONDS);
        driver.get("http://demo.guru99.com/V4/");
        objLogin.setUserName("mngr559943");
        objLogin.setPassword("AqajysY");
        objLogin.clickLogin();
        driver.findElement(By.linkText("Fund Transfer")).click();
    }

    @Test(priority=1, description="Valid transfer")
    public void test_Valid_Fund_Transfer() {
        objTransfer.setPayerAccount("153279");
        objTransfer.setPayeeAccount("153280");
        objTransfer.setAmount("100");
        objTransfer.setDescription("Test transfer");
        objTransfer.clickSubmit();
        Assert.assertTrue(
            objTransfer.getResultMessage().contains("Fund Transfer Successfully"),
            "Transfer should have succeeded");
    }

    @Test(priority=2, description="Invalid account — expect error")
    public void test_Invalid_Fund_Transfer() {
        objTransfer.setPayerAccount("000000");   // invalid account
        objTransfer.setPayeeAccount("000001");
        objTransfer.setAmount("100");
        objTransfer.setDescription("Invalid test");
        objTransfer.clickSubmit();
        // AjaxElementLocatorFactory: waits 15 s, then NoSuchElementException
        // if success element never appears — we assert the error message instead
        Assert.assertFalse(
            objTransfer.getResultMessage().contains("Fund Transfer Successfully"),
            "Transfer should have failed for invalid account");
    }

    @AfterMethod
    public void tearDown() { driver.quit(); }
}
```

Thank You

Software Quality Assurance

Lab 7 · POM & Page Factory